

Mtech Software Engineering

Full Stack Application Development (S2-25_SEZG503)



By Salandri Shirisha: 2025TM93131

Objective:

To design and develop a complete full-stack web application using modern frontend and backend technologies. The project aims to demonstrate practical implementation of REST APIs, database integration, authentication, role-based access control, and responsive user interface development. It also focuses on integrating frontend and backend components into a fully functional system while following modern software engineering practices.



Name of the Application: *Smart Shipment Management System*

Tagline: *Delivering Logistics with Intelligence and Efficiency*

Theme Name: *Modern Enterprise Logistics Dashboard*

Theme Style: *White and light blue color palette*

1. Introduction:

The rapid growth of global trade and logistics has increased the need for efficient shipment management systems. Traditional shipment tracking and management processes often rely on manual operations, spreadsheets, emails, and disconnected software systems. These methods lead to delays, poor visibility, operational inefficiencies, and increased human error.

To address these challenges, a modern full-stack web application called the **Smart Shipment Management System** was developed. The system provides an end-to-end platform for managing shipment requests, approvals, employee assignments, transportation details, and delivery status updates.

The application is built using modern web technologies including React, TypeScript, Tailwind CSS, Spring Boot, MySQL, and JWT-based authentication. The system supports multiple user roles including administrators, employees, and clients.

2. Problem Statement:

Logistics and shipment operations in many organizations are still managed using manual records, spreadsheets, and disconnected systems, which leads to delays, poor tracking, approval issues, and inefficient communication between clients, administrators, and employees. There is a need for a centralized digital platform that can efficiently manage shipment requests, approvals, employee assignments, and shipment tracking in a secure and organized manner.

The proposed **Smart Shipment Management System** is a full-stack web application designed to streamline shipment management processes through role-based access, real-time status updates, secure authentication, dashboard monitoring, and efficient workflow management for admins, clients, and employees.

3. Objectives of the System:

The major objectives of the Smart Shipment Management System are:

1. To provide a centralized shipment management platform.

2. To automate shipment request creation and approval workflows.
3. To implement secure JWT-based authentication.
4. To provide role-based access control.
5. To maintain shipment records using a relational database.
6. To improve operational efficiency.
7. To build a responsive and modern user interface.
8. To integrate frontend and backend systems using REST APIs.
9. To explore AI-assisted software development.
10. To provide scalable architecture for future improvements.

4. Proposed Solution:

The proposed solution is a full-stack web application that provides:

- Secure user authentication
- Role-based dashboards
- Shipment creation and approval
- Employee assignment
- Shipment tracking
- Pagination and search functionality
- RESTful API integration
- Responsive UI
- Database persistence using MySQL

The system follows modern software engineering practices including modular architecture, layered backend design, reusable frontend components, and secure API communication.

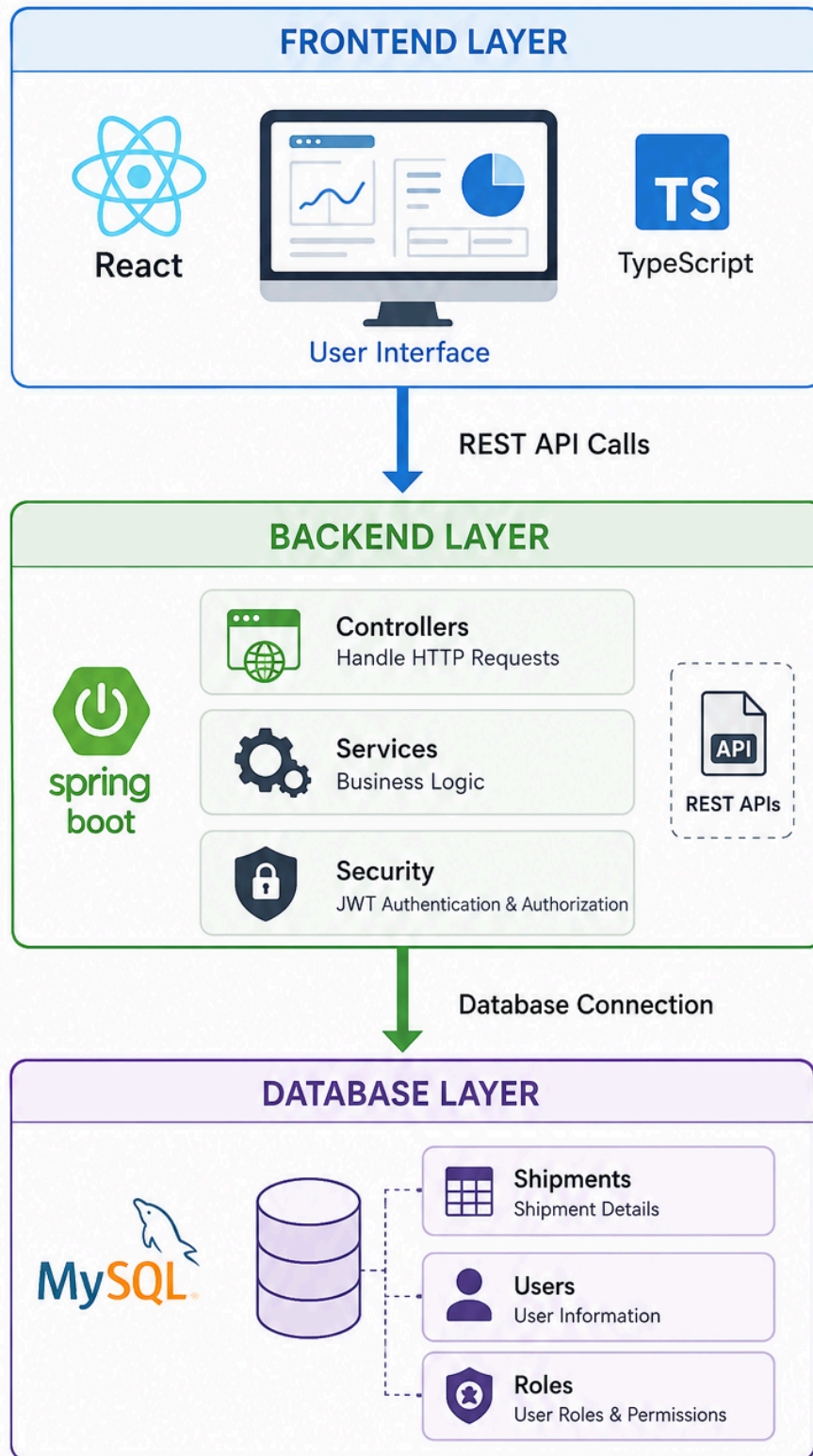
5. Software Architecture:

The project follows a layered architecture.

- Frontend Layer: Built using React, TypeScript, and Tailwind CSS.
- Backend Layer: Built using Spring Boot REST APIs.
- Database Layer: MySQL database for data persistence.
- Security Layer: JWT-based authentication and authorization.

SMART SHIPMENT MANAGEMENT SYSTEM

HIGH-LEVEL ARCHITECTURE DIAGRAM



Backend Internal Architecture:

Controller Layer



Service Layer



Repository Layer



Database

Backend Technologies:

Spring Boot: Used for REST API development.

Spring Security: Used for authentication and authorization.

JWT: Used for token-based authentication.

JPA/Hibernate: ORM framework for database operations.

Database:

MySQL: Relational database used for data persistence.

6. Backend Development:

The backend was developed using Spring Boot.

Backend Modules

Controller Layer

Handles incoming HTTP requests.

Example:

- ShipmentController
- AuthController
- AdminController

Service Layer

Contains business logic.

Example:

- ShipmentService
- UserService
- AuthService

Repository Layer

Handles database communication using JPA repositories.

Entity Layer: Defines database models.

DTO Layer: Used for request and response transfer.

Backend Features

- CRUD operations
- Pagination support
- Search functionality
- Role-based authorization
- JWT token validation
- Exception handling
- Secure password encryption

7. Database Design:

- Main Tables
- Users Table

Stores user information.

Fields:

- user_id
- name
- email
- password
- role

Shipments Table

Stores shipment information.

Fields:

- shipment_id
- shipment_type
- origin_country
- destination_country
- pickup_address
- delivery_address
- item_name
- quantity
- weight
- transport_mode
- status
- created_date

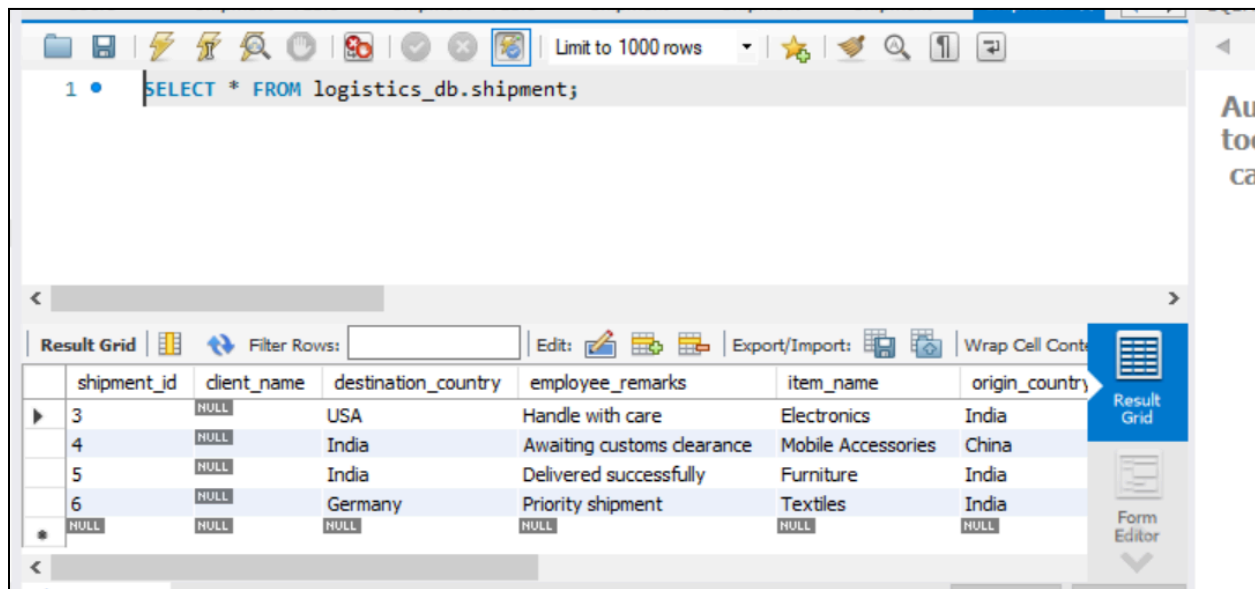
Employee Assignment: Stores shipment assignment

The screenshot shows a database management interface with a query editor and a result grid. The query editor contains the SQL statement: `SELECT * FROM logistics_db.users;`. The result grid displays the following data:

user_id	email	name	password	role
7	shiri@gmail.com	shiri	\$2a\$10\$PkWkaRB880FoD7J6jqMs2eHtJWYy3...	CLIENT
8	admin@gmail.com	admin	\$2a\$10\$yepES4GDi9E1xjEt2oqvh./0eShs1.dQB...	ADMIN
9	salandri@gmail.com	salandri	\$2a\$10\$UBU9SHUjd6BAqjnm2ARGOeAJ0Tteeh...	EMPLOYEE
10	satya@gmail.com	satya	\$2a\$10\$DmD5QL26XZBy8nTzuzSmceNE./RlzJ7...	CLIENT
11	nirusha@gmail.com	Nirusha	\$2a\$10\$nB2htg1OSDRIJQpXbv.QwOITvbOcan...	CLIENT
12	riya@gmail.com	riya	\$2a\$10\$a0B8rjRc70aODjvuucY.ZCldo7s6.YUC...	EMPLOYEE

Below the result grid, there is an 'Output' section showing a log of actions and their results:

#	Time	Action	Message
9	16:33:06	SELECT * FROM logistics_db.users LIMIT 0, 1000	2 row(s) returned
10	21:53:14	SELECT * FROM logistics_db.shipment LIMIT 0, 1000	4 row(s) returned
11	21:53:15	SELECT * FROM logistics_db.shipment LIMIT 0, 1000	4 row(s) returned
12	21:53:16	SELECT * FROM logistics_db.shipment LIMIT 0, 1000	4 row(s) returned
13	21:53:16	SELECT * FROM logistics_db.shipment LIMIT 0, 1000	4 row(s) returned
14	21:54:07	SELECT * FROM logistics_db.users LIMIT 0, 1000	7 row(s) returned



The screenshot shows a database management interface. At the top, there is a toolbar with various icons and a text input field containing the SQL query: `SELECT * FROM logistics_db.shipment;`. Below the query, a "Result Grid" is displayed, showing a table with 7 columns: `shipment_id`, `client_name`, `destination_country`, `employee_remarks`, `item_name`, and `origin_country`. The table contains 6 rows of data. The first row has `shipment_id` 3, `client_name` NULL, `destination_country` USA, `employee_remarks` "Handle with care", `item_name` "Electronics", and `origin_country` "India". The second row has `shipment_id` 4, `client_name` NULL, `destination_country` "India", `employee_remarks` "Awaiting customs clearance", `item_name` "Mobile Accessories", and `origin_country` "China". The third row has `shipment_id` 5, `client_name` NULL, `destination_country` "India", `employee_remarks` "Delivered successfully", `item_name` "Furniture", and `origin_country` "India". The fourth row has `shipment_id` 6, `client_name` NULL, `destination_country` "Germany", `employee_remarks` "Priority shipment", `item_name` "Textiles", and `origin_country` "India". The fifth row has `shipment_id` NULL, `client_name` NULL, `destination_country` NULL, `employee_remarks` NULL, `item_name` NULL, and `origin_country` NULL. The interface also includes a "Filter Rows" field, an "Edit" button, an "Export/Import" button, and a "Wrap Cell Content" button. On the right side, there are buttons for "Result Grid" and "Form Editor".

	shipment_id	client_name	destination_country	employee_remarks	item_name	origin_country
▶	3	NULL	USA	Handle with care	Electronics	India
	4	NULL	India	Awaiting customs clearance	Mobile Accessories	China
	5	NULL	India	Delivered successfully	Furniture	India
	6	NULL	Germany	Priority shipment	Textiles	India
*	NULL	NULL	NULL	NULL	NULL	NULL

Employee Assignment: Stores shipment assignments.

8. API Design and Endpoints

Authentication APIs

Login API :POST /api/auth/login

Register API: POST /api/auth/register

Shipment APIs

Create Shipment: POST /api/shipments

Get Shipments: GET /api/shipments

Update Shipment: PUT /api/shipments/{id}

Delete Shipment: DELETE /api/shipments/{id}

Admin APIs

Approve Shipment: PUT /api/admin/approve/{id}

Reject Shipment: PUT /api/admin/reject/{id}

localhost:8080/swagger-ui/index.html#

Servers
http://localhost:8080 - Generated server url

Authorize

shipment-controller

- GET /api/shipments/{id}
- PUT /api/shipments/{id}
- DELETE /api/shipments/{id}
- GET /api/shipments
- POST /api/shipments

employee-controller

- PUT /api/employee/update-status/{shipmentId}
- GET /api/employee/shipments

client-controller

- PUT /api/client/update-shipment/{shipmentId}
- POST /api/client/create-shipment
- GET /api/client/my-shipments
- DELETE /api/client/delete-shipment/{shipmentId}

client-controller

- PUT /api/client/update-shipment/{shipmentId}
- POST /api/client/create-shipment
- GET /api/client/my-shipments
- DELETE /api/client/delete-shipment/{shipmentId}

admin-controller

- PUT /api/admin/update-user/{id}
- PUT /api/admin/reject/{shipmentId}
- PUT /api/admin/assign
- PUT /api/admin/approve/{shipmentId}
- POST /api/admin/create-employee
- POST /api/admin/create-client
- GET /api/admin/employees
- GET /api/admin/clients
- DELETE /api/admin/delete-user/{id}

auth-controller ^		
POST	/api/auth/register	🔒 ▼
POST	/api/auth/login	🔒 ▼
dashboard-controller ^		
GET	/api/dashboard/employee/{employeeId}	🔒 ▼
GET	/api/dashboard/client/{clientId}	🔒 ▼
GET	/api/dashboard/admin	🔒 ▼
Schemas ^		
Shipment >		
User >		
LoginRequest >		
PageShipment >		
PageableObject >		

Error Handling and Validation: Proper error handling improves reliability.

Validation Features

- Required field validation
- Email validation
- Input sanitization
- Duplicate email checks

Exception Handling: Global exception handling is implemented using:

@ControllerAdvice

Common Errors

- Invalid credentials
- Unauthorized access
- Resource not found
- Validation failure
- Internal server error

9. Frontend Development:

The frontend was developed using React, Tailwind CSS, and TypeScript.

Frontend Goals

- Responsive design
- Reusable components
- Clean UI

- Dashboard visualization
- Fast API integration

Component-Based Architecture: Components created:

- Navbar
- Sidebar
- DataTable
- StatusBadge
- Loader
- Pagination
- DashboardCards

9.1 Frontend Pages and Features:

Login Page Features:

- Email/password authentication
- JWT storage
- Validation messages
- Protected navigation

Dashboard Page Features:

- Shipment statistics
- Role-based overview
- Quick navigation

Admin Shipment Page Features:

- Shipment listing
- Pagination
- Search functionality
- Shipment approval
- Employee assignment

Client Dashboard Features:

- Create shipment
- View shipment history
- Track shipment status

Employee Dashboard Features:

- Assigned shipment list
- Status updates
- Remarks management

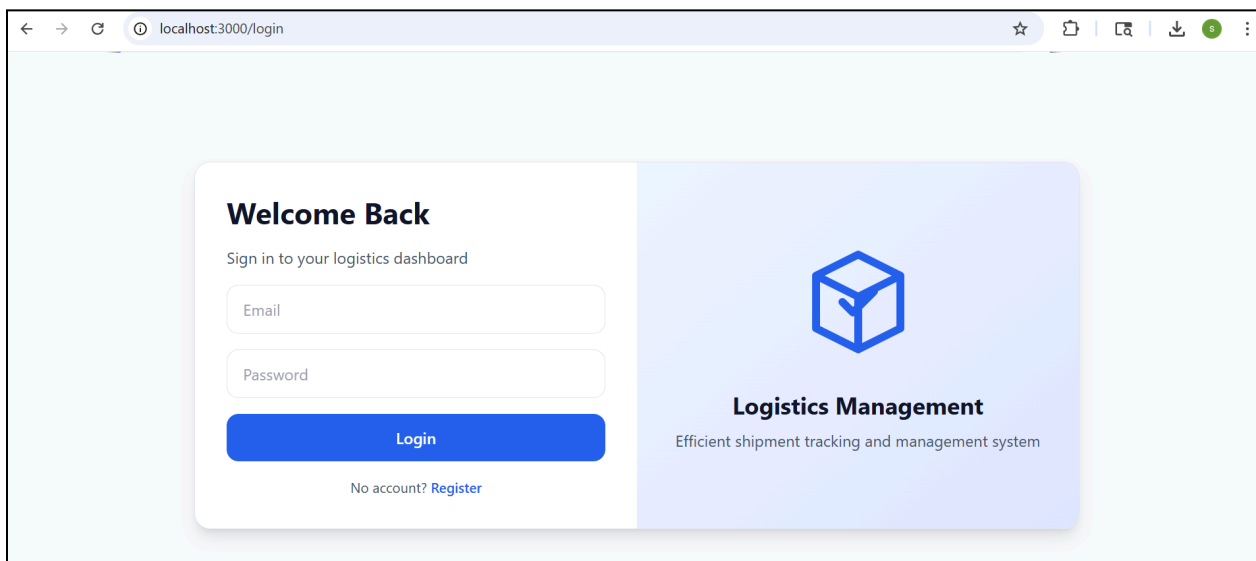
10. Application Overview

The Smart Shipment Management System is divided into three major modules:

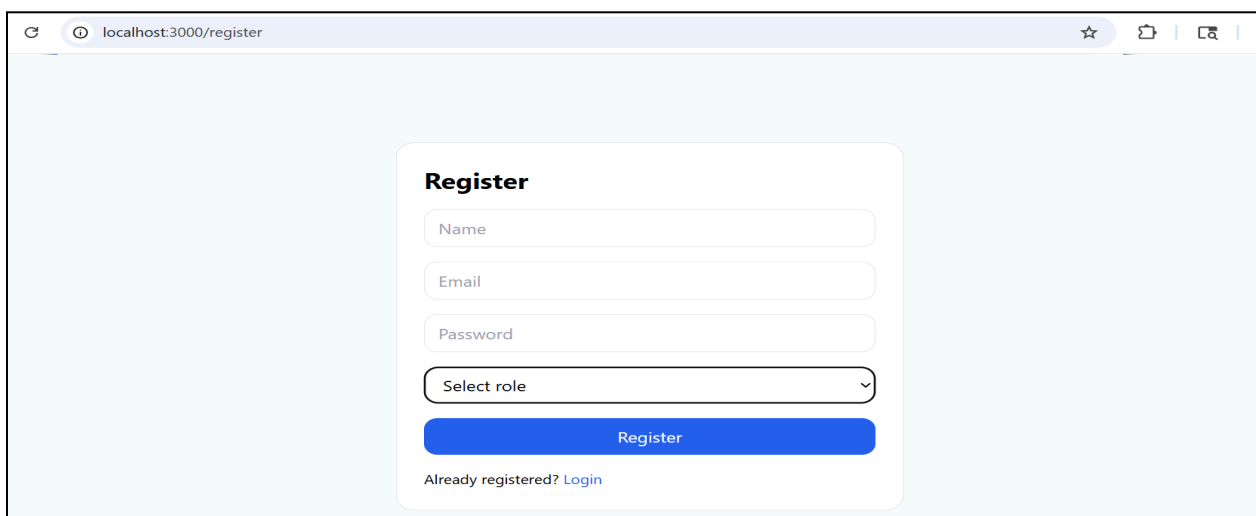
1. Client Module

Clients can:

- Register and login
- Create shipment requests
- Track shipment status
- View shipment history



The screenshot shows a web browser window with the address bar displaying 'localhost:3000/login'. The page features a light blue background. On the left, a white card contains the heading 'Welcome Back' and the text 'Sign in to your logistics dashboard'. Below this are input fields for 'Email' and 'Password', followed by a blue 'Login' button. At the bottom of the card, it says 'No account? [Register](#)'. On the right, a light blue card displays a blue cube icon with a white arrow, the heading 'Logistics Management', and the text 'Efficient shipment tracking and management system'.



The screenshot shows a web browser window with the address bar displaying 'localhost:3000/register'. The page features a light blue background. In the center, a white card contains the heading 'Register'. Below this are input fields for 'Name', 'Email', and 'Password', followed by a dropdown menu labeled 'Select role'. A blue 'Register' button is positioned below the dropdown. At the bottom of the card, it says 'Already registered? [Login](#)'.

Logistics Panel

Dashboard

My Shipments

Create Shipment

Edit Shipment

Delete Shipment

Logout

Client Dashboard

Welcome back! Here's your shipment overview.

Total Shipments

5

Pending

4

Approved

1

Delivered

0

Recent Shipments

Your latest shipment activity

ID	Item	Qty	Weight (kg)	Origin	Destination	Mode	Assigned To	Status
2	Steel Pipes	67	2	USA	India	ROAD	salandri	approved
3	Electronics	10	120.5	India	USA	AIR	Not Assigned	PENDING
4	Mobile Accessories	50	300	China	India	SEA	Not Assigned	PENDING
5	Furniture	5	500	India	India	ROAD	Not Assigned	PENDING
6	Textiles	25	200	India	Germany	AIR	Not Assigned	PENDING

Logistics Panel

Dashboard

My Shipments

Create Shipment

Edit Shipment

Delete Shipment

Logout

Shipment Logistics Management

shiri (CLIENT)

My Shipments

ID	Item	Origin	Destination	Status	Actions
2	Steel Pipes	USA	India	approved	EditDelete
3	Electronics	India	USA	PENDING	EditDelete
4	Mobile Accessories	China	India	PENDING	EditDelete
5	Furniture	India	India	PENDING	EditDelete
6	Textiles	India	Germany	PENDING	EditDelete

Shipment Logistics Management

Create Shipment

IMPORT

▼

Origin Country

Destination Country

Pickup Address

Delivery Address

Item Name

Quantity

Weight

Transport Mode

Create Shipment

Edit Shipment

Select a shipment and update its details.

ID	Item
2	Steel Pipes
3	Electronics
4	Mobile Accessories
5	Furniture
6	Textiles

Edit Shipment #2

Close

USA

India

Dallas

Hyderabaddddy

Steel Pipes

67

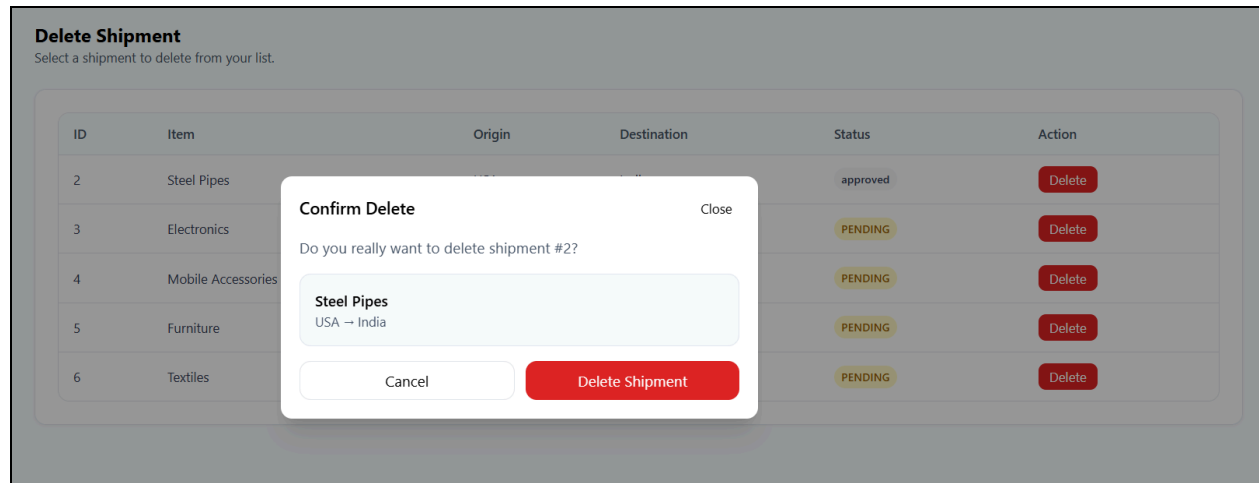
2

ROAD

Cancel

Save Changes

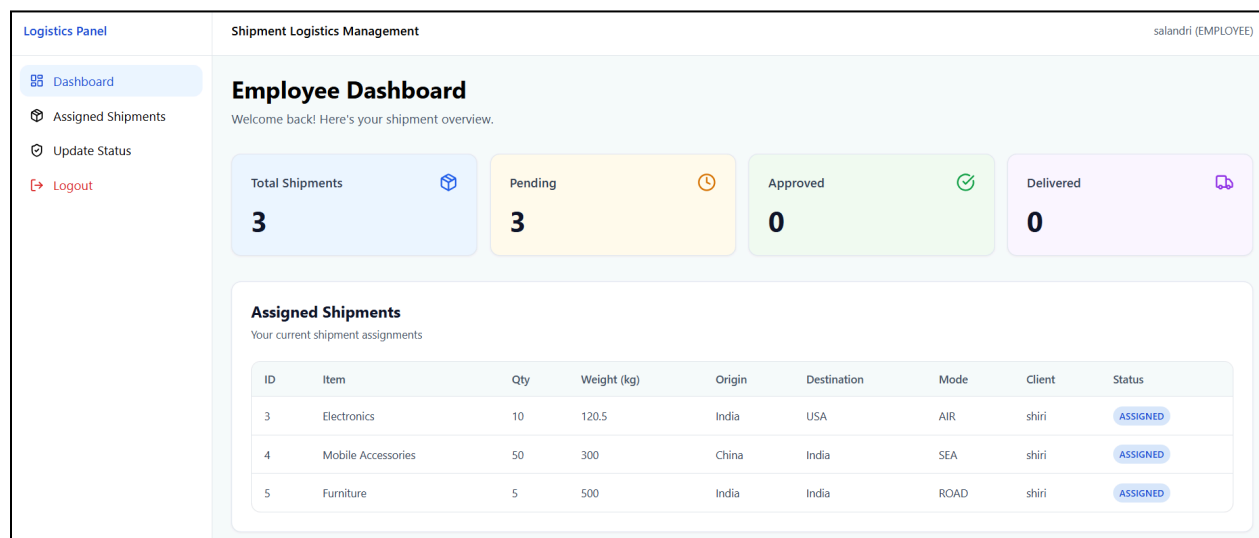
Status	Action
approved	Edit
PENDING	Edit
PENDING	Edit
PENDING	Edit
PENDING	Edit



2. Employee Module

Employees can:

- View assigned shipments
- Update shipment statuses
- Add remarks
- Manage shipment operations



Assigned Shipments

ID	Item	Mode	Type	Status
3	Electronics	AIR	EXPORT	ASSIGNED
4	Mobile Accessories	SEA	IMPORT	ASSIGNED
5	Furniture	ROAD	DOMESTIC	ASSIGNED

Update Shipment Status

Select a shipment and update its status

Assigned Shipments

ID	Item	Qty	Weight (kg)	Origin	Destination	Mode	Client	Status	Action
3	Electronics	10	120.5	India	USA	AIR	shiri	ASSIGNED	Select
4	Mobile Accessories	50	300	China	India	SEA	shiri	ASSIGNED	Select
5	Furniture	5	500	India	India	ROAD	shiri	ASSIGNED	Select

Assigned Shipments

ID	Item	Qty	Weight (kg)	Origin	Destination	Mode	Client	Status	Action
3	Electronics	10	120.5	India	USA	AIR	shiri	ASSIGNED	Selected
4	Mobile Accessories	50	300	China	India	SEA	shiri	ASSIGNED	Select
5	Furniture	5	500	India	India	ROAD	shiri	ASSIGNED	Select

Update Status for Shipment #3

Status

APPROVED

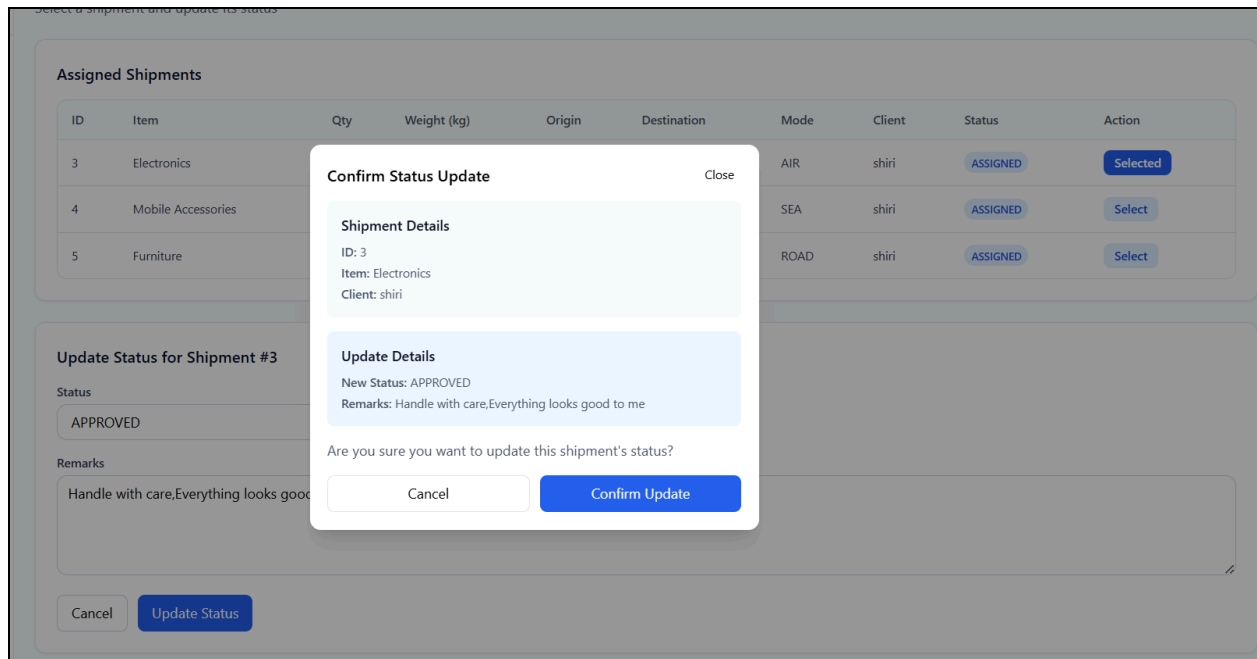


Remarks

Handle with care,Everything looks good to me

Cancel

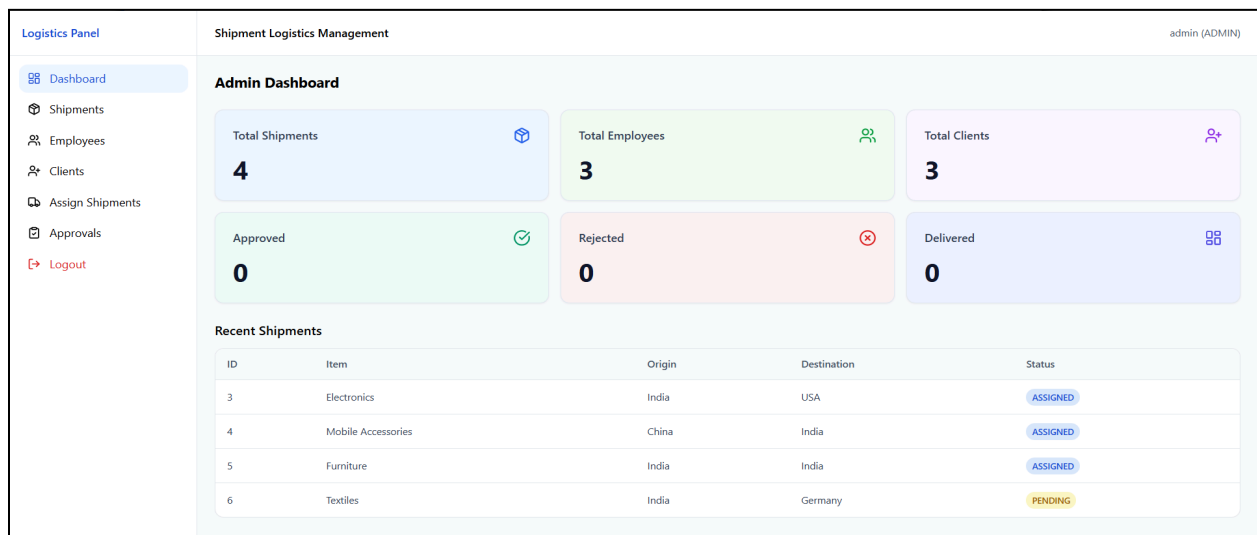
Update Status



3. Admin Module

Administrators can:

- View all shipments
- Approve or reject shipments
- Assign employees
- Manage users
- Monitor operations



Shipment Logistics Management

admin (ADMIN)

All Shipments

Q Search shipments...

Search

ID	Item	Client	Mode	Status	Actions
3	Electronics	shiri	AIR	ASSIGNED	ApproveReject
4	Mobile Accessories	shiri	SEA	ASSIGNED	ApproveReject
5	Furniture	shiri	ROAD	ASSIGNED	ApproveReject
6	Textiles	shiri	AIR	PENDING	ApproveReject

Shipment Logistics Management

Employees

ID	Name	Email
9	salandri	salandri@gmail.com
12	riya	riya@gmail.com
13	Yash	yash@gmail.com

Shipment Logistics Management

Clients

ID	Name	Email
7	shiri	shiri@gmail.com
10	satya	satya@gmail.com
11	Nirusha	nirusha@gmail.com

Shipment Logistics Management

admin (ADMIN)

Assign Shipments

ID	Item	Client	Mode	Status	Actions
3	Electronics	shiri	AIR	ASSIGNED	<button>Assign</button>
4	Mobile Accessories	shiri	SEA	ASSIGNED	<button>Assign</button>
5	Furniture	shiri	ROAD	ASSIGNED	<button>Assign</button>
6	Textiles	shiri	AIR	PENDING	<button>Assign</button>

shiri

AIR

ASSIGNED

ASSIGNED

ASSIGNED

PENDING

es

Select Employee

Close

salandri

salandri@gmail.com

riya

riya@gmail.com

Yash

yash@gmail.com

Cancel

Confirm Assign

Logistics Panel	Shipment Logistics Management
- Dashboard - Shipments - Employees - Clients - Assign Shipments Approvals - Logout	Approvals Shipment ID Approve Reject

11 Functional Requirements:

User Authentication

- User registration
- Login functionality
- JWT token generation
- Secure route protection

Shipment Management

- Create shipment
- Update shipment
- Delete shipment
- View shipment details

Role-Based Access

- Admin access
- Employee access
- Client access

Search and Pagination

- Shipment search
- Pagination for large datasets

Dashboard Features

- Statistics display
- Shipment overview
- Recent activities

11.1 Non-Functional Requirements:

Security

The system must implement JWT authentication and encrypted password storage.

Scalability

The application should support future feature expansion.

Performance

API response time should remain optimized.

Reliability

Database transactions must maintain data consistency.

Usability

The UI should be responsive and user-friendly.

12. Technology Stack:

Frontend Technologies

React

React was used for building reusable UI components and SPA architecture.

TypeScript

Type safety and better maintainability.

Tailwind CSS

Utility-first CSS framework for responsive design.

React Router

Used for navigation and route protection.

Axios

Used for API communication.

13. Integration Between Frontend and Backend:

The frontend communicates with backend APIs using Axios.

API Integration Process

1. React component sends request.
2. Axios attaches JWT tokens.
3. Backend validates the token.
4. Database operation performed.
5. Response returned to frontend.
6. UI updates dynamically.

Features Implemented

- Dynamic rendering
- API response handling
- Loading states
- Error notifications

14. Testing and Debugging:

Backend Testing

- API testing using Postman
- Validation testing
- Authentication testing

Frontend Testing

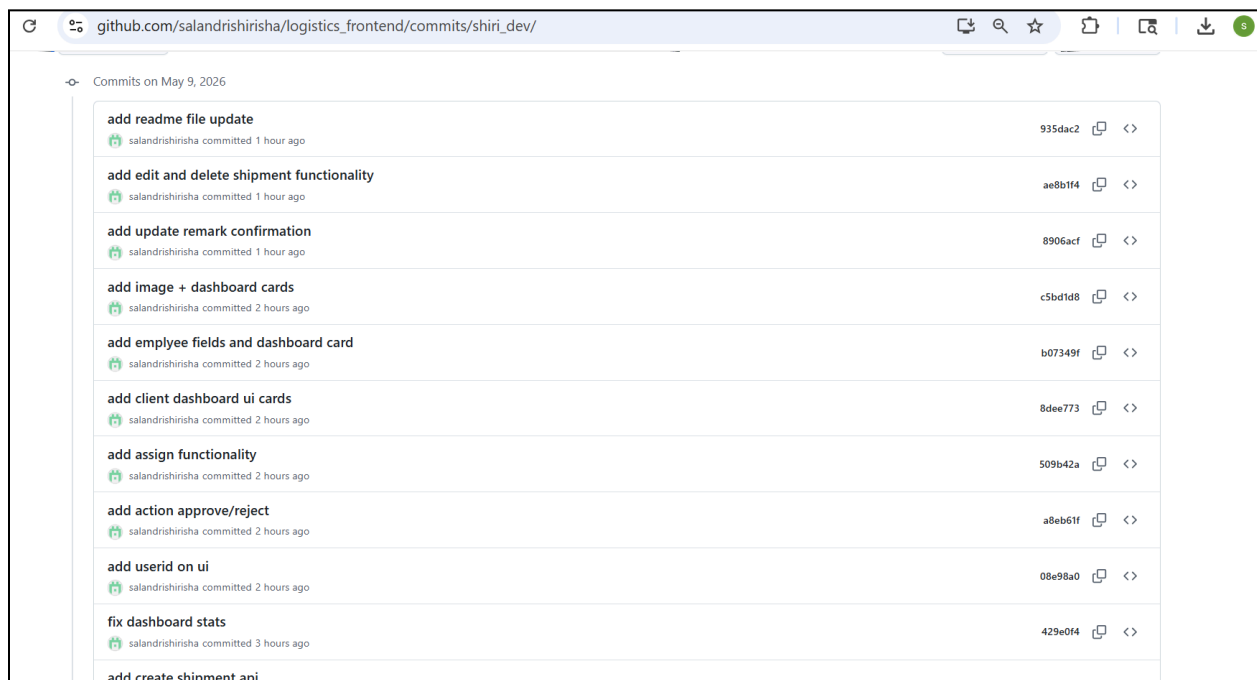
- UI testing
- Route testing
- Form validation testing

Debugging

Common debugging tools used:

- Browser developer tools
- Console logging
- Postman testing
- Spring Boot logs

Code Quality, GIT commit History, Browser testing



Commits on May 9, 2026		
add readme file update	935dac2	<>
salandrishirisha committed 1 hour ago		
add edit and delete shipment functionality	ae8b1f4	<>
salandrishirisha committed 1 hour ago		
add update remark confirmation	8906acf	<>
salandrishirisha committed 1 hour ago		
add image + dashboard cards	c5bd1d8	<>
salandrishirisha committed 2 hours ago		
add employee fields and dashboard card	b07349f	<>
salandrishirisha committed 2 hours ago		
add client dashboard ui cards	8dee773	<>
salandrishirisha committed 2 hours ago		
add assign functionality	509b42a	<>
salandrishirisha committed 2 hours ago		
add action approve/reject	a8eb61f	<>
salandrishirisha committed 2 hours ago		
add userid on ui	08e98a0	<>
salandrishirisha committed 2 hours ago		
fix dashboard stats	429e0f4	<>
salandrishirisha committed 3 hours ago		
add create shipment api		

logistics_frontendPublic

shiri_dev2 Branches0 Tags

Go to file

Add fileCode

This branch is 18 commits ahead of main

salandrishirishaadd readme file update935dac2 · 1 hour ago19 Commits

public	Initial setup	7 hours ago
src	add edit and delete shipment functionality	1 hour ago
.gitignore	Initial setup	7 hours ago
README.md	add readme file update	1 hour ago
eslint.config.js	Initial setup	7 hours ago
index.html	add stly setup	7 hours ago
package-lock.json	role based route access	5 hours ago
package.json	role based route access	5 hours ago
postcss.config.js	role based route access	5 hours ago
tailwind.config.js	role based route access	5 hours ago
tsconfig.app.json	Initial setup	7 hours ago
tsconfig.json	Initial setup	7 hours ago
tsconfig.node.json	Initial setup	7 hours ago
vite.config.ts	add stly setup	7 hours ago

About

No description provided.

ReadmeActivity0 stars0 watching0 forks

Releases

No releases publishedCreate a new release

Packages

No packages publishedPublish your first package

Contributors

salandrishirisha

Languages

CSS 50.3%JavaScript

Suggested

Based on your repository

salandrishirisha / logistics

IssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.

logisticsPublic

shiri_dev had recent pushes 10 seconds agoCompare & pull request

main2 Branches0 Tags

Go to file

Add fileCode

salandrishirishaInitial setup5e63114 · 9 hours ago1 Commit

.mvnw/wrapper	Initial setup	9 hours ago
src	Initial setup	9 hours ago
.gitattributes	Initial setup	9 hours ago
.gitignore	Initial setup	9 hours ago
mvnw	Initial setup	9 hours ago
mvnw.cmd	Initial setup	9 hours ago
pom.xml	Initial setup	9 hours ago

README

About

No description, website, or topics provided.

Activity0 stars0 watching0 forks

Releases

No releases publishedCreate a new release

Packages

No packages publishedPublish your first package

Contributors1

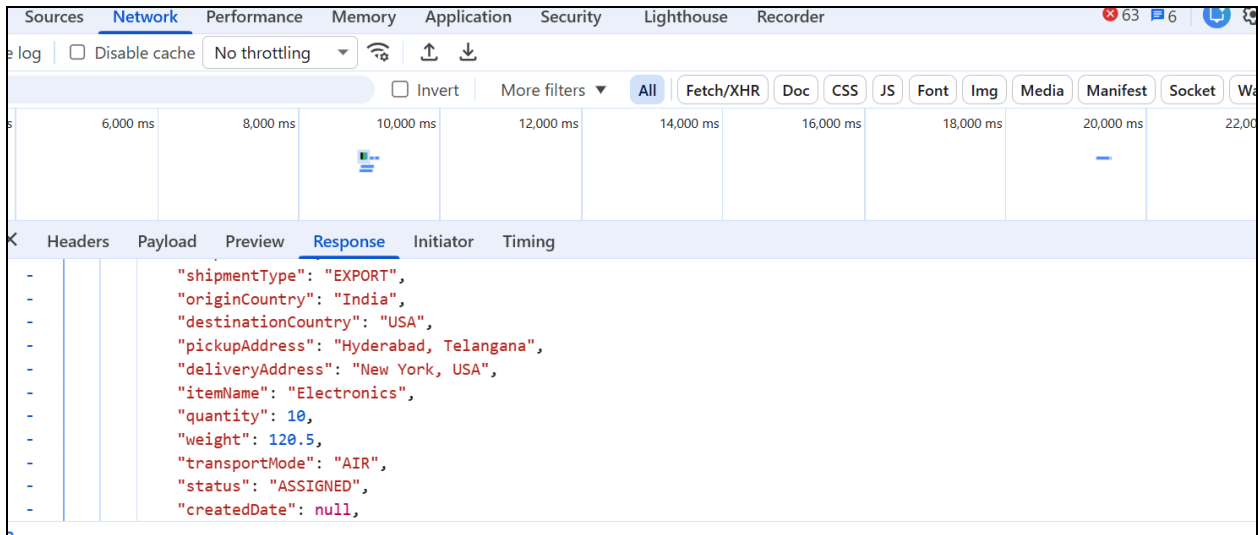
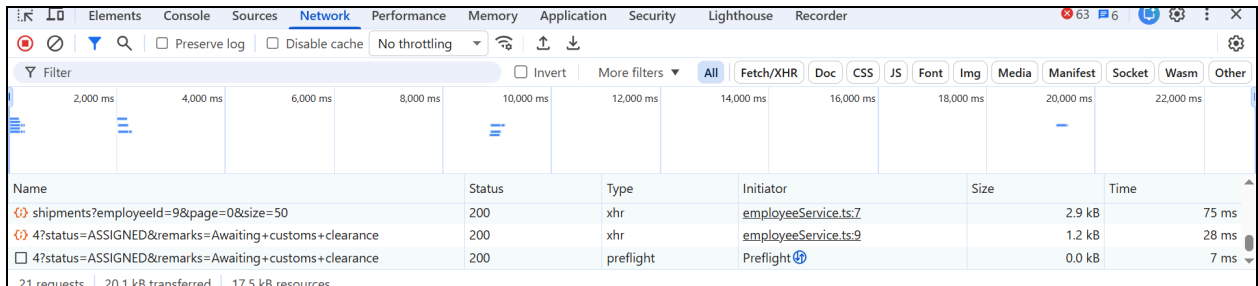
salandrishirishasalandrishirisha

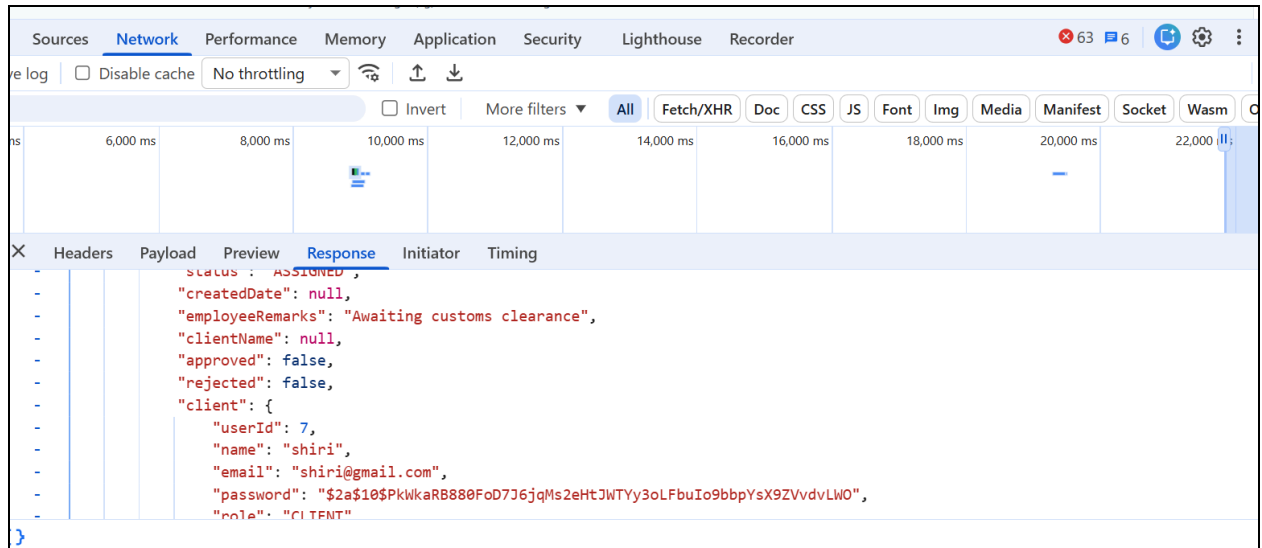
Commits

shiri_dev
 🔍 All users
📅 All time

Commits on May 9, 2026

add edit and delet client shipment salandrishirisha committed now	fae389e
add dashboard based role salandrishirisha committed 1 hour ago	6ac4204
add cors access salandrishirisha committed 5 hours ago	5e747d7
add maven depend salandrishirisha committed 8 hours ago	22d7849
add jwt token salandrishirisha committed 9 hours ago	b2e3cb4
add EXception setup salandrishirisha committed 9 hours ago	a13885c
Initial setup salandrishirisha committed 9 hours ago	5a63114





15. AI-Assisted Development:

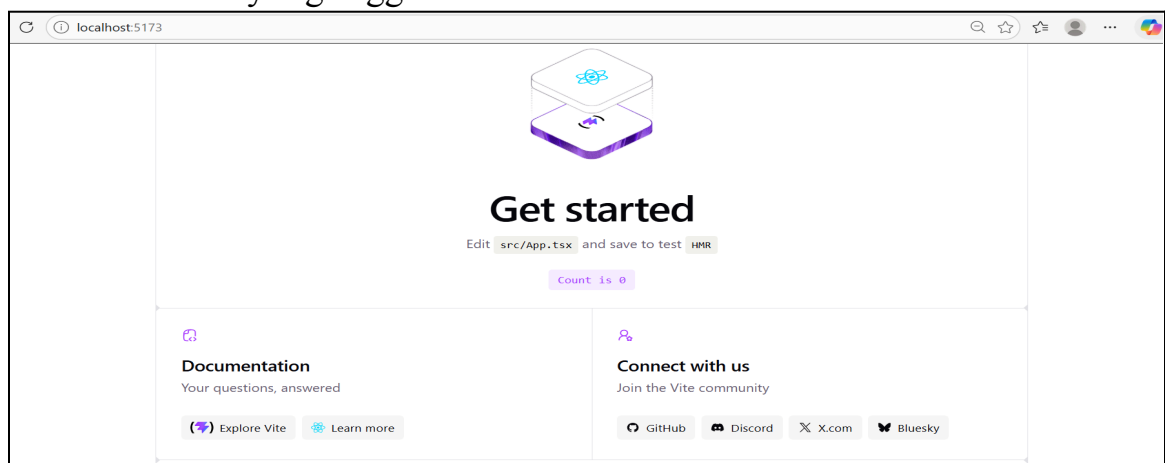
AI-assisted development tools were extensively used during project implementation.

AI Tools Used

GitHub Copilot

Used for:

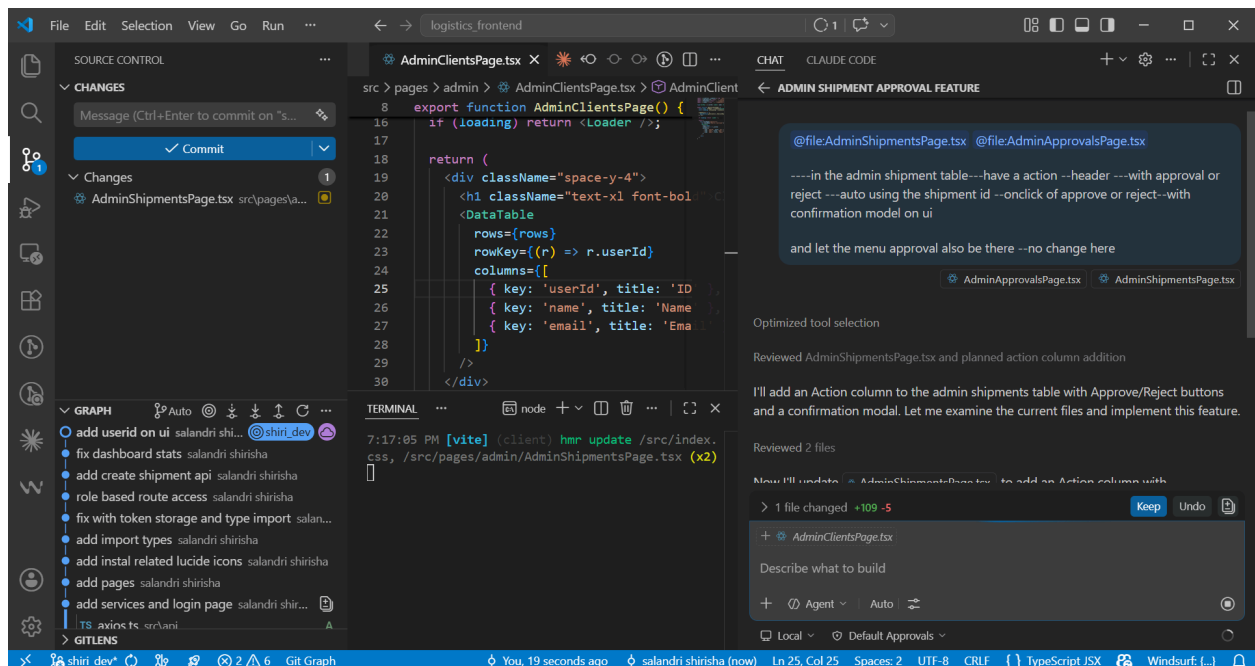
- Boilerplate code generation
- React component suggestions
- API integration support
- Tailwind styling suggestions

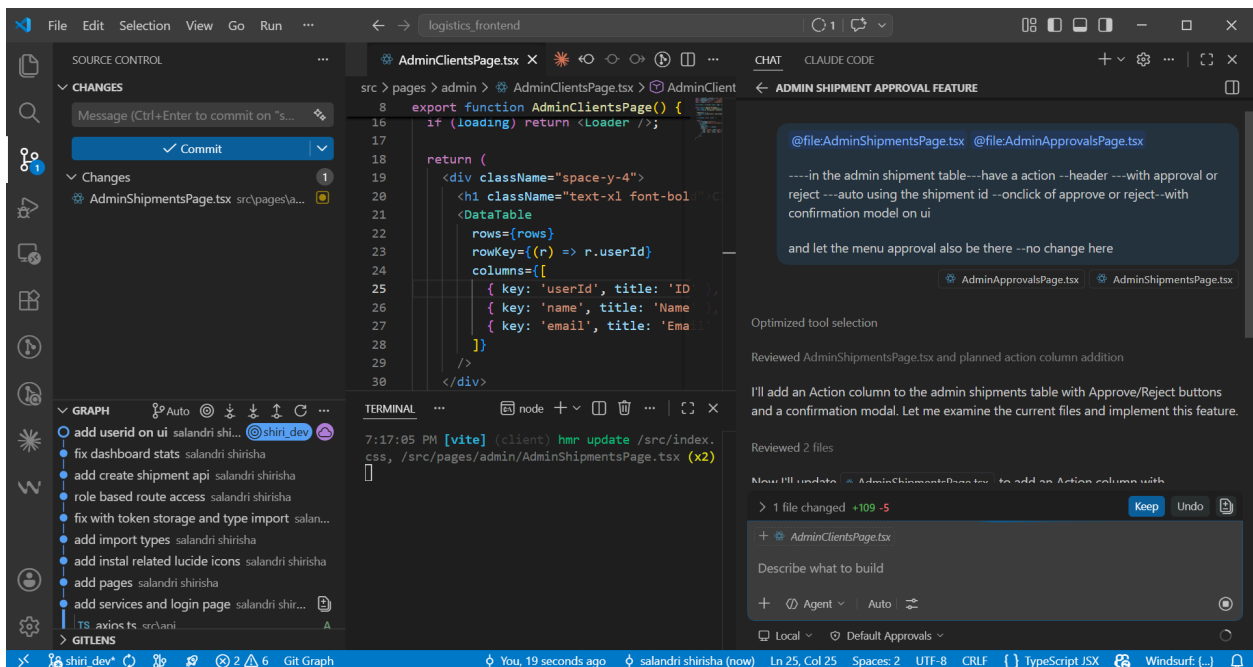
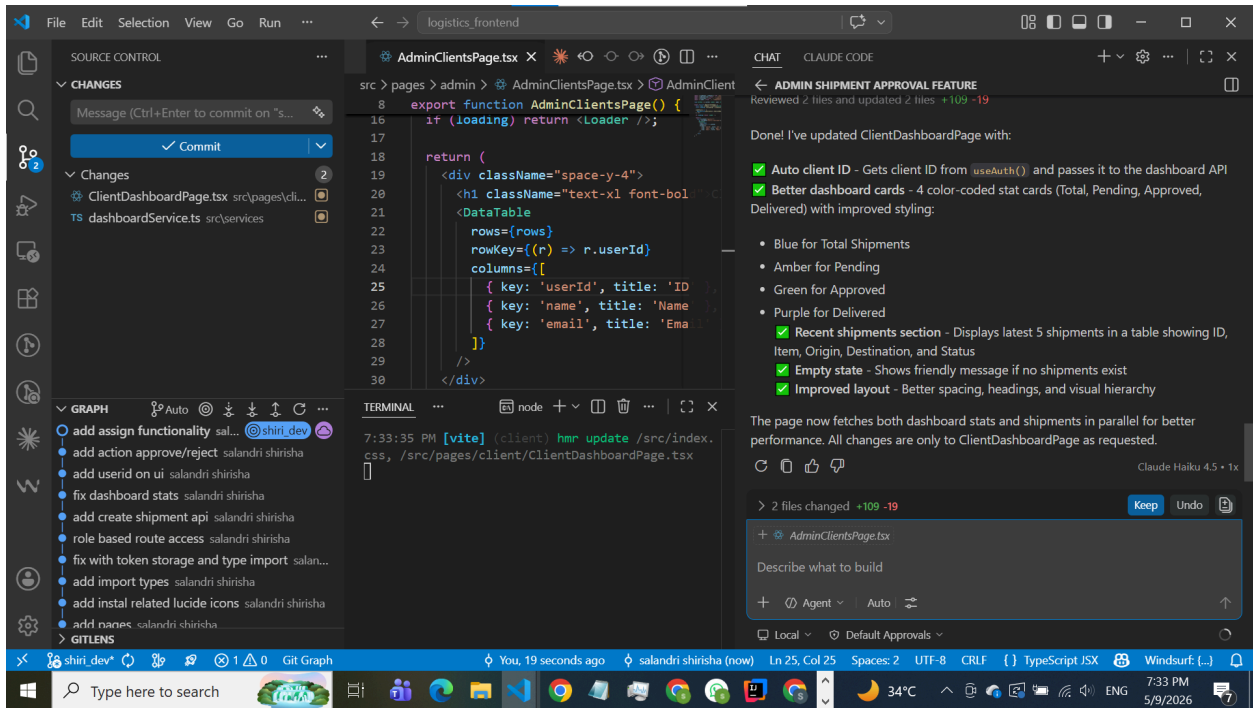


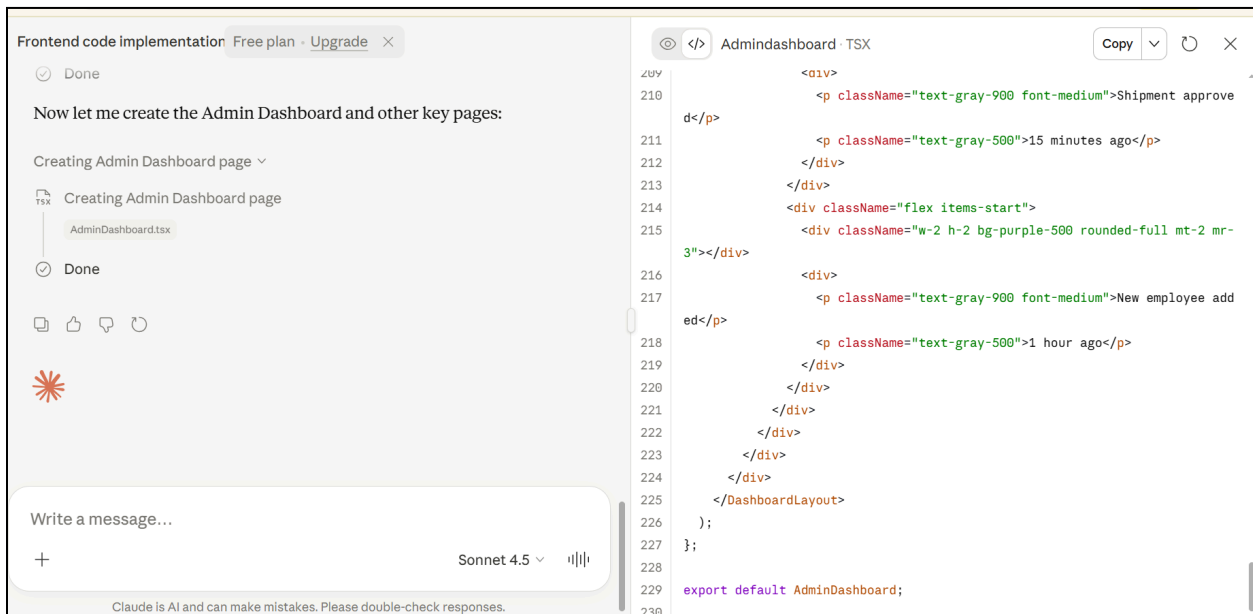
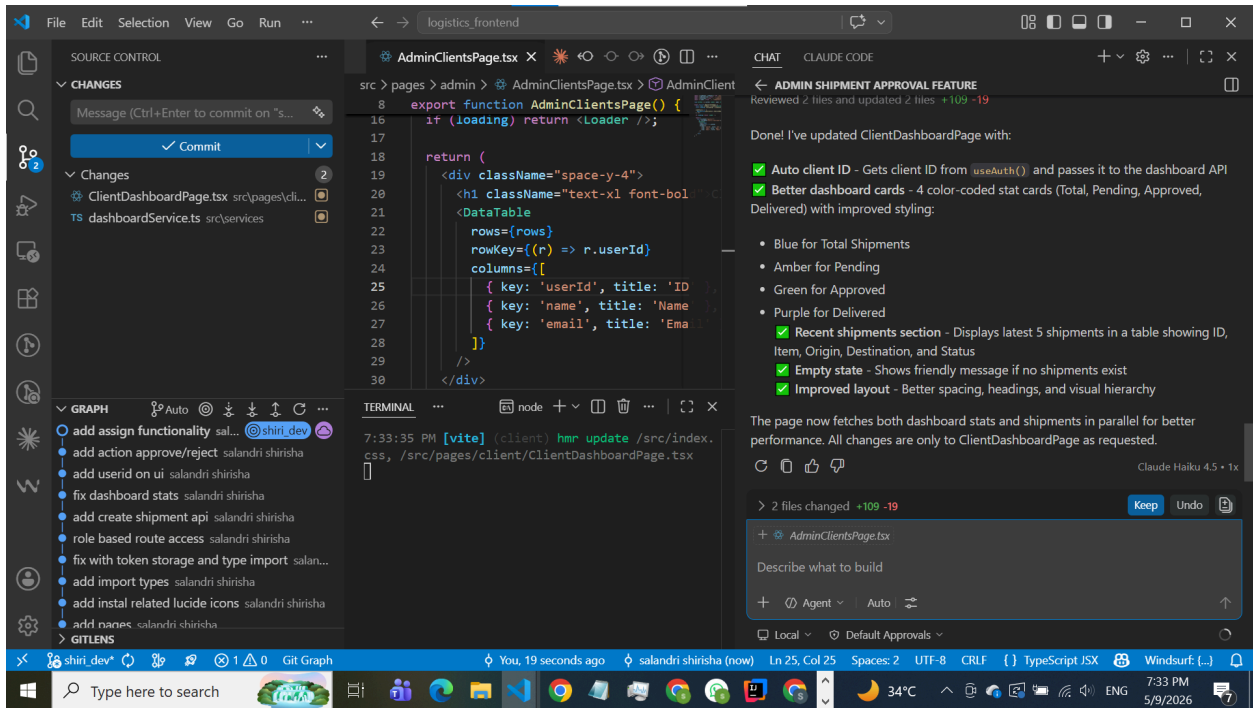
Claude AI

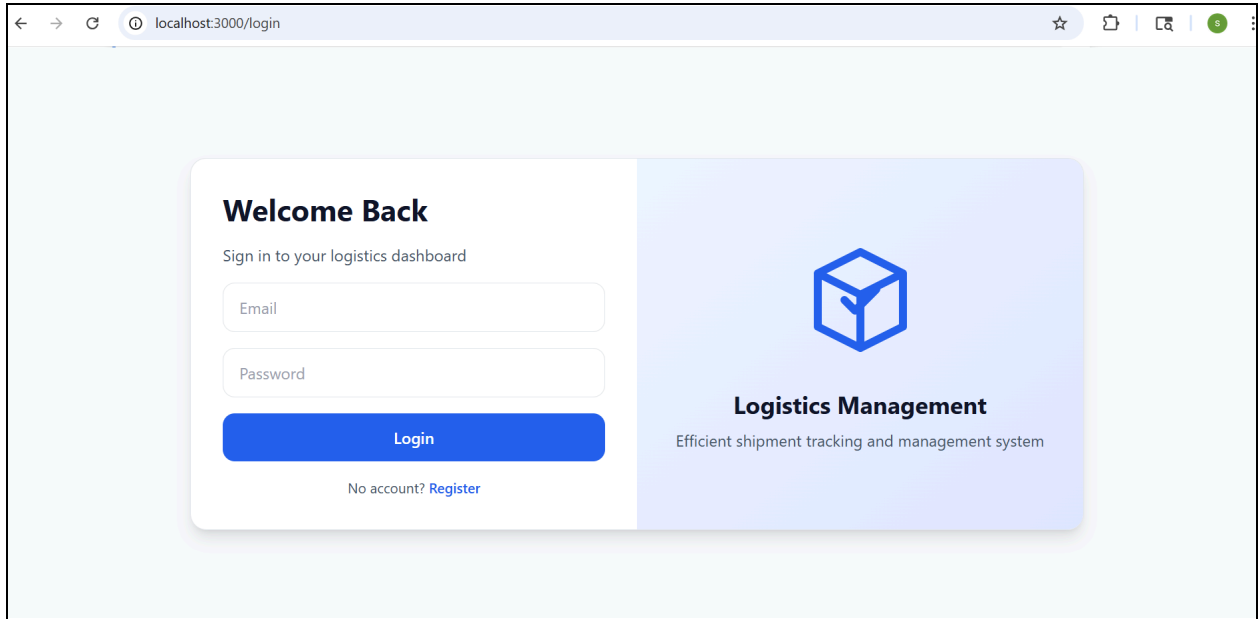
Used for:

- UI improvement ideas
- Error debugging
- TypeScript support
- Backend troubleshooting









Copilot Free

Upgrade

Chat messages

24% used

Resets Jun 1

Inline Suggestions

0% used

Resets Jun 1

Inline Suggestions >

Disabled

Codebase Semantic Index >

⚠ Index not yet built

16. AI Usage Reflection:

Artificial Intelligence tools were actively used during the development of the Smart Shipment Management System to improve productivity, accelerate frontend development, and assist in debugging. The project followed a hybrid approach where core business logic and architecture decisions were implemented manually, while AI tools were used mainly for boilerplate generation, UI suggestions, and development assistance.

The primary AI tools used were:

- GitHub Copilot
- Claude AI

GitHub Copilot Usage

GitHub Copilot was mainly used for generating repetitive and boilerplate code.

Areas Where Copilot Helped

- React component templates
- TypeScript interfaces
- Tailwind CSS layouts
- Axios API request structures
- Dashboard UI components
- Pagination and table components
- Form structures and reusable components

Copilot also helped generate built-in reusable UI components such as:

- DataTable
- StatusBadge
- Loader
- Pagination
- Sidebar and Navbar layouts

These suggestions reduced development time and provided a strong base structure for implementation.

Claude AI Usage:

Claude AI was mainly used for:

- UI enhancement ideas
- Debugging TypeScript issues
- Backend response handling
- Frontend validation improvements
- API integration troubleshooting
- Error handling improvements

Claude AI was especially useful when adjusting frontend validations according to backend error responses and improving overall user experience.

Manual Development Responsibilities

Although AI tools assisted development, important application logic was manually implemented.

Manually Managed Features

- JWT token authentication flow
- Session storage management
- Axios API setup
- Protected routing
- Role-based access control
- Backend integration
- State management
- Business logic implementation
- Shipment workflow handling

The complete frontend-backend integration and role management system were manually configured and tested.

Reflection:

AI tools significantly improved development speed and reduced repetitive coding effort. However, all generated code required manual validation, debugging, and customization.

Benefits Observed

Faster Development

AI reduced time spent on:

- UI setup
- Boilerplate coding
- TypeScript definitions
- API structure creation
- Tailwind styling

Improved UI Development

AI-generated suggestions helped create:

- Responsive dashboards
- Better layout structures
- Reusable components
- Cleaner Tailwind designs

Better Debugging Support

Claude AI helped resolve:

- React rendering issues
- TypeScript errors
- API response mismatches
- Validation handling problems

Example AI Workflow Prompt

“Create a React shipment management page using TypeScript and Tailwind CSS following the existing dashboard structure with pagination, search, and API integration.”

AI Generated

- Initial page structure
- Table layout
- Tailwind responsive UI
- Basic API integration structure

Manual Improvements:

The generated code was manually refined by:

- Connecting real backend APIs
- Adding JWT authentication
- Implementing role validation
- Handling backend errors
- Optimizing state management
- Refactoring unnecessary code

Security Considerations:

Sensitive configuration files were never shared with AI tools.

The following were kept private:

- .env files
- Database credentials
- JWT secret keys
- API secrets

Only safe development-level code snippets were used for AI assistance.

Limitations of AI

Some challenges observed while using AI tools included:

- Overly complex code generation
- Redundant logic
- Generic implementations
- Extra unnecessary lines of code
- Incorrect assumptions about APIs. In several cases, manually written code was simpler and more optimized than AI-generated solutions.

All AI-generated code required:

- Manual review
- Refactoring
- Debugging
- Testing

AI tools significantly improved development speed and productivity.

Benefits Observed

- Reduced repetitive coding
- Faster debugging
- Better UI suggestions
- Improved code structure
- Quick API documentation generation

16.1 Challenges Faced:

JWT Integration Issues

Managing token expiration and protected routes required careful implementation.

CORS Errors

Frontend-backend communication initially caused CORS configuration issues.

Role-Based Routing

Implementing secure route protection required debugging.

Pagination Logic

Synchronizing backend pagination with frontend UI required adjustments.

AI Generated Code Errors

Some AI-generated code required optimization and corrections.

17. Conclusion:

The Smart Shipment Management System successfully demonstrates the implementation of a modern full-stack web application using React, TypeScript, Tailwind CSS, Spring Boot, MySQL, and JWT authentication.

The system automates shipment management workflows including shipment creation, approval, assignment, and tracking. Role-based access ensures secure operations for administrators, employees, and clients.

The project also highlights the importance of AI-assisted software development using GitHub Copilot and Claude AI. These tools accelerated development, reduced repetitive coding effort, and improved productivity.

Through this project, valuable experience was gained in:

- Full-stack web development
- REST API integration
- Secure authentication systems
- Database management
- Responsive UI design
- AI-assisted software engineering

The project fulfills all assignment objectives and demonstrates practical implementation of industry-standard software development practices.

CODE:

Frontend: https://github.com/salandrishirisha/logistics_frontend

Backend: <https://github.com/salandrishirisha/logistics>

Demo_Link: <https://drive.google.com/file/d/1z0fd896r6seysYS4s-Kbhlv0nSAES0mc/view?usp=sharing>

Thank you!